HDOC DAY - 57

Algorithm

1. Start.
2. Read the size of the array.
3. Declare a variable-length integer array arr[size].
4. Read the elements of the array.
5. Pass the array address and size to max_subarray_sum().
6. Initialize current_sum and max_sum with the first element.
7. Traverse the array from the second element.
8. For each element:
    - Compare the current element with the sum of current_sum and the current element.
    - Store the larger value in current_sum.
    - If current_sum is greater than max_sum, update max_sum.
9. Display max_sum.
10. Stop.

C program:

```
#include <stdio.h>


void max_subarray_sum(int *arr, int size)
{
   int current_sum = *arr;
   int max_sum = *arr;
   int i;
```

```
    for (i = 1; i < size; i++)
    {
        if (current_sum + *(arr + i) > *(arr + i))
            current_sum = current_sum + *(arr + i);
        else
            current_sum = *(arr + i);

        if (current_sum > max_sum)
            max_sum = current_sum;
    }

    printf("Maximum subarray sum: %d\n", max_sum);
}

int main()
{
    int size, i;

    printf("Enter the size of the array: ");
    scanf("%d", &size);

    int arr[size];

    printf("Enter the array elements: ");
```

```
    for (i = 0; i < size; i++)
        scanf("%d", &arr[i]);

    max_subarray_sum(arr, size);

    return 0;
}
```

Test case table:

Test Case	Array	Expected Output
1	-2 1 -3 4 -1 2 1 -5 4	Maximum subarray sum: 6
2	-8 -3 -6 -2 -5 -4	Maximum subarray sum: -2
3	5 -2 3 4	Maximum subarray sum: 10
4	1 2 3 4 5	Maximum subarray sum: 15
5	-5	Maximum subarray sum: -5